GATI SHAKTI VISHWAVIDYALAYA

MLOps LAB ASSIGNMENT

LAB 6–7

House Price Prediction using ML Pipeline, FastAPI and Docker

Submitted by

Chevakulla Sidhartha

Roll Number: 24AI019

B.Tech – Artificial Intelligence and Data Science

Abstract

This project demonstrates an end-to-end Machine Learning Operations (MLOps) workflow for house price prediction. A Random Forest Regression model is trained using house features such as area, number of bedrooms, house age and location score. The workflow is organized into separate Python modules for data loading and validation, model training, evaluation and prediction. The trained model is exposed through a FastAPI application, tested using Pytest, and packaged with Docker. Docker Compose is used to define and start the application as a service. The local and containerized applications were tested with the same input and returned the same prediction.

1. Introduction

Machine Learning projects involve more than training a model. A complete workflow also needs data checks, repeatable training, evaluation, prediction interfaces, testing and deployment. MLOps combines these activities so that a machine learning application can be organized, tested and run consistently.

In this assignment, a house price prediction application was developed. The model uses four numerical input features and predicts the house price. The application is served through a REST API built with FastAPI and deployed in a Docker container.

2. Objectives

- Create a modular machine learning project structure.
- Load and validate the house price dataset.
- Split the data into training and testing sets.
- Train and save a Random Forest Regression pipeline.
- Evaluate the model using MAE, RMSE and R^2 score.
- Create a prediction API with health and prediction endpoints.
- Write automated tests for the data pipeline and API.
- Build and run the application using Docker.
- Use Docker Compose to define and start the application.
- Compare local and Docker predictions using the same input.

3. Tools and Technologies

Technology	Purpose
Python 3.11	Main programming language and Docker base image
Pandas	Loading and handling tabular data
Scikit-learn	Train/test split, preprocessing pipeline and Random Forest model
Joblib	Saving and loading the trained model
PyYAML	Reading model and project settings from config.yaml
FastAPI	Creating REST API endpoints
Pydantic	Validating API input values
Pytest	Running automated tests
Docker	Packaging and running the API in a container
Docker Compose	Defining and starting the application service

Git and GitHub	Version control and remote project repository

4. Dataset Description

The project uses dataset.csv stored in data/raw/. The dataset contains 33 records and five columns. Four columns are used as input features and the price column is the target variable.

Column	Meaning	Role
area_sqft	Area of the house in square feet	Input feature
bedrooms	Number of bedrooms	Input feature
age_years	Age of the house in years	Input feature
location_score	Numerical location score	Input feature
price	House price	Target

The data was split into 80% training data and 20% testing data using random_state=42. This produced 26 training records and 7 testing records. The fixed random state helps make the split repeatable.

5. Project Structure

```
MLOps-Lab-6-7/
├── configs/
│   └── config.yaml
├── data/
│   ├── raw/
│   │   └── dataset.csv
│   └── processed/
├── models/
│   └── house_price_pipeline.pkl
├── reports/
│   └── evaluation_report.json
├── src/
│   ├── __init__.py
│   ├── data.py
│   ├── train.py
│   ├── evaluate.py
│   ├── predict.py
│   └── api.py
├── tests/
│   ├── test_api.py
│   └── test_pipeline.py
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
├── .dockerignore
├── .gitignore
└── README.md
```

Main module responsibilities:

- data.py: loads the CSV file, checks required data conditions and splits features from the target.
- train.py: reads configuration, builds and trains the machine learning pipeline, evaluates it and saves the model.
- evaluate.py: loads the saved model, calculates evaluation metrics and writes the report JSON file.
- predict.py: loads the trained model and predicts a price for supplied house details.
- api.py: defines the FastAPI application, input validation, health endpoint and prediction endpoint.

- tests/: contains automated tests for the API and the data/model pipeline.

6. Configuration and Model Training

The file configs/config.yaml stores the dataset path, target column, test size, random state, model settings and output paths. Keeping these settings in one configuration file makes it easier to change parameters without editing the main training code.

```
data:
  path: data/raw/dataset.csv
  target: price
  test_size: 0.2
  random_state: 42

model:
  name: RandomForestRegressor
  n_estimators: 100
  random_state: 42

output:
  model_path: models/house_price_pipeline.pkl
  report_path: reports/evaluation_report.json
```

The training module uses a Scikit-learn Pipeline with a SimpleImputer and RandomForestRegressor. The pipeline is fitted on the training data and saved as models/house_price_pipeline.pkl using Joblib.

Training command:

```
python -m src.train
```

The training run completed successfully. It used 26 training rows and 7 testing rows. The saved model is then reused by the prediction module and the API.

7. Model Evaluation

The model was evaluated using three regression metrics. MAE measures the average absolute difference between actual and predicted prices. RMSE gives more weight to larger errors. R^2 compares the model with a simple baseline that predicts the average target value.

Metric	Observed result
MAE	1,518,485.71
RMSE	3,972,086.71
R^2 Score	-0.1352

The negative R^2 score indicates that, on this test split, the model performed worse than predicting the mean target value. The dataset is small (33 records), so the evaluation result may be unstable and is not evidence of a reliable real-world price estimator. More representative data, data-quality review and model tuning would be needed to improve performance.

Evaluation command:

```
python -m src.evaluate
```

The calculated metrics are saved in reports/evaluation_report.json.

8. Prediction Module

The prediction module loads the saved pipeline and accepts four input values: area, bedrooms, age and location score. It creates a one-row DataFrame with the same feature names used during training and calls the model's predict method.

Prediction command:

```
python -m src.predict
```

The script generated predictions for five example house inputs. The output values varied substantially, which is consistent with the limited performance observed in the evaluation. These predictions are included as a demonstration of the prediction workflow and should not be treated as reliable valuations.

9. FastAPI Application

FastAPI was used to make the trained model available through HTTP endpoints. The application loads the saved model when the API module starts. Pydantic validates the incoming request fields before a prediction is made.

9.1 Health endpoint

Method: GET Endpoint: /health

This endpoint returns a healthy status message when the API is running.

9.2 Prediction endpoint

Method: POST Endpoint: /predict

This endpoint accepts the four house features and returns the predicted price. Input validation requires positive area and bedroom count, non-negative house age, and a location score between 0 and 10.

```
{
  "area_sqft": 1000,
  "bedrooms": 2,
  "age_years": 10,
  "location_score": 7
}
```

The API can be started locally with:

```
uvicorn src.api:app --reload
```

Interactive API documentation is available at http://127.0.0.1:8000/docs.

10. Automated Testing

Pytest was used to check the data pipeline and API. The tests cover dataset loading, data validation, data splitting, saved model existence, the health endpoint, a valid prediction request and an invalid prediction request.

```
python -m pytest -v
```

Test	Result
Health endpoint	**PASSED**
Valid prediction request	**PASSED**
Invalid prediction request	**PASSED**
Dataset loading	**PASSED**
Data validation	**PASSED**

Data splitting	**PASSED**
Saved model exists	**PASSED**

Final test result: 7 passed, 1 warning. The warning was a Starlette deprecation warning related to the test client dependency. It did not cause any test to fail.

11. Docker Deployment

A Dockerfile was created to package the application. It uses the python:3.11-slim base image, sets /app as the working directory, installs requirements, copies the project files, exposes port 8000 and starts the API using Uvicorn.

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "src.api:app", "--host", "0.0.0.0", "--port", "8000"]
```

The Docker image was built with:

```
docker build -t house-price-api .
```

The container was initially run with:

```
docker run -d -p 8000:8000 --name house-price-container house-price-api
```

The container was checked using docker ps. The API was accessed through http://127.0.0.1:8000/docs and its health and prediction endpoints were tested.

12. Local and Docker Prediction Comparison

The same input was passed to the local prediction function and to the Docker-hosted POST /predict endpoint:

```
{
  "area_sqft": 1000,
  "bedrooms": 2,
  "age_years": 10,
  "location_score": 7
}
```

The local prediction was 465100.0. The Docker API returned the same predicted price, 465100.0. This confirms matching output for this particular test input. It does not by itself establish that the model is accurate for unseen houses.

13. Docker Compose

Docker Compose was configured in docker-compose.yml to build the application image, create a container named house-price-compose and map host port 8000 to container port 8000.

```
services:
 house-price-api:
  build: .
  container_name: house-price-compose
  ports:
   - "8000:8000"
```

The service was started using:

```
docker compose up --build -d
```

The running service was verified using:

```
docker compose ps
```

The command output showed the house-price-compose container in the Up state with port 8000 mapped. The API health endpoint was also tested after starting it with Docker Compose.

14. Git and GitHub

Git was initialized in the project directory. The project files were staged and committed, the local main branch was connected to the remote repository, and the commit was pushed successfully.

GitHub repository:

https://github.com/Drushadva07/MLOps-Lab-6-7

The push output confirmed that the main branch was created on the remote repository and configured to track origin/main.

15. Screenshots

Insert the screenshots captured during implementation into the marked spaces below. Use clear screenshots where the command, result or API response is readable. The placeholders are provided so the report can be completed with the user's actual screenshots rather than illustrative or fabricated images.

Figure 1. Project folder structure

[INSERT SCREENSHOT HERE]
01_project_structure.png

Show the project folders and Python modules in VS Code.

Figure 2. Configuration file

[INSERT SCREENSHOT HERE]
02_config_yaml.png

Show the dataset path, target, split and model settings.

Figure 3. Model training output

[INSERT SCREENSHOT HERE]
03_training_output.png

Show successful training, saved model path and training/test row counts.

Figure 4. Evaluation results

[INSERT SCREENSHOT HERE]
04_evaluation_metrics.png

Show MAE, RMSE and R^2 results.

Figure 5. Prediction output

[INSERT SCREENSHOT HERE]
05_prediction_output.png

Show the sample predictions produced by src.predict.

Figure 6. FastAPI health endpoint

[INSERT SCREENSHOT HERE]
06_fastapi_health.png

Show GET /health with a successful response.

Figure 7. FastAPI prediction endpoint

[INSERT SCREENSHOT HERE]
07_fastapi_predict.png

Show POST /predict with the sample request and response.

Figure 8. Automated test results

[INSERT SCREENSHOT HERE]
08_pytest_results.png

Show the final Pytest output with 7 passed.

Figure 9. Docker image build

[INSERT SCREENSHOT HERE]
09_docker_build.png

Show the successful docker build output.

Figure 10. Docker Compose service

[INSERT SCREENSHOT HERE]
10_docker_compose_ps.png

Show docker compose ps with house-price-compose Up.

Figure 11. Docker Compose health test

[INSERT SCREENSHOT HERE]
11_compose_health.png

Show the successful health response while Compose is running.

Figure 12. Local and Docker prediction match

[INSERT SCREENSHOT HERE]
12_prediction_comparison.png

Show the local result and Docker API response for the same input.

16. Challenges and Observations

- After a computer shutdown, Docker Desktop needed to be opened again. The existing image and container were retained, and the container could be started again.
- The original container occupied host port 8000, so it was stopped before starting the Docker Compose service on the same port.
- Git displayed a line-ending warning for config.yaml (LF and CRLF). This was a warning and did not prevent staging or pushing the project.
- The model's negative R^2 score and large errors showed that the small dataset limits prediction quality.

17. Conclusion

The assignment implemented an end-to-end MLOps workflow for a house price prediction application. The work included modular data handling, model training and evaluation, saved-model prediction, FastAPI endpoints, automated tests, Docker packaging and Docker Compose deployment. All seven automated tests passed. The local and Docker applications returned the same prediction for the tested input, and the Compose service started successfully. The evaluation also showed that the current model has limited predictive performance because the

dataset is small. Overall, the project demonstrates the main steps required to package, test and deploy a machine learning application.

Appendix: Useful Commands

- python -m src.train
- python -m src.evaluate
- python -m src.predict
- python -m pytest -v
- uvicorn src.api:app --reload
- docker build -t house-price-api .
- docker ps
- docker compose up --build -d
- docker compose ps
- docker compose down